Algorithms and Problem Solving

CSCI 1101 Introduction to Computer Science

Definition of Algorithm

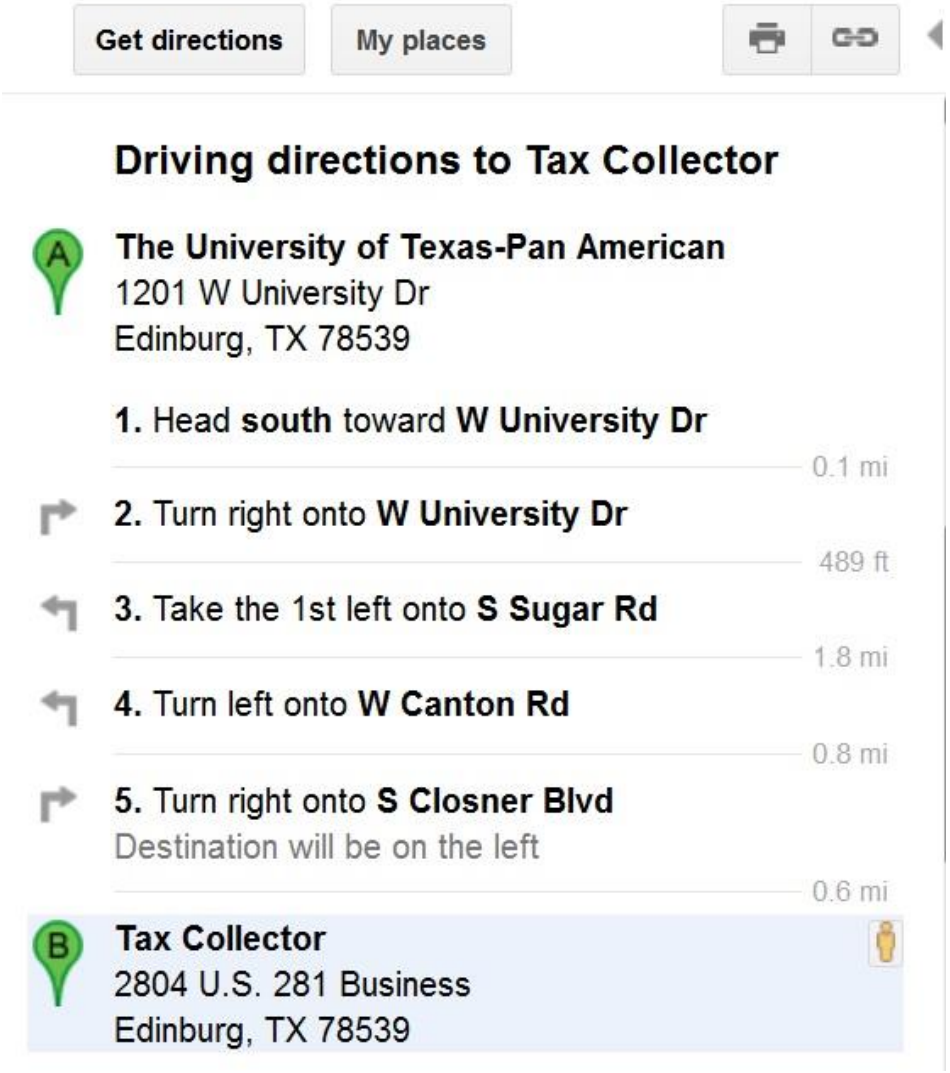
An algorithm is an **ordered** set of **unambiguous**, **executable** steps that defines a **terminating** process.

Ambiguous statement: please check the air pressure of the front left tire.
Unambiguous statement: please check the air pressure of the front driver's side tire.

Algorithm Representation

- Requires well-defined primitives
- A collection of primitives constitutes a programming language

A simple algorithm



Google Maps primitives

- Head south
- Turn right
- Turn left
- Take the ramp
- Take the exit
- Keep right
- Merge onto
- . . .

Pseudocode

- **Pseudocode:**
  - A notational system for **informally** expressing algorithms
  - Must be clear and consistent
  - Should be programming language **independent** (*i.e.*, a programmer should be able to use any programming language to implement the pseudocode's algorithm)

Pseudocode Primitives

- Assignment

  set *name* **to** *expression*

- Conditional selection

  - **if** *condition* **then** action

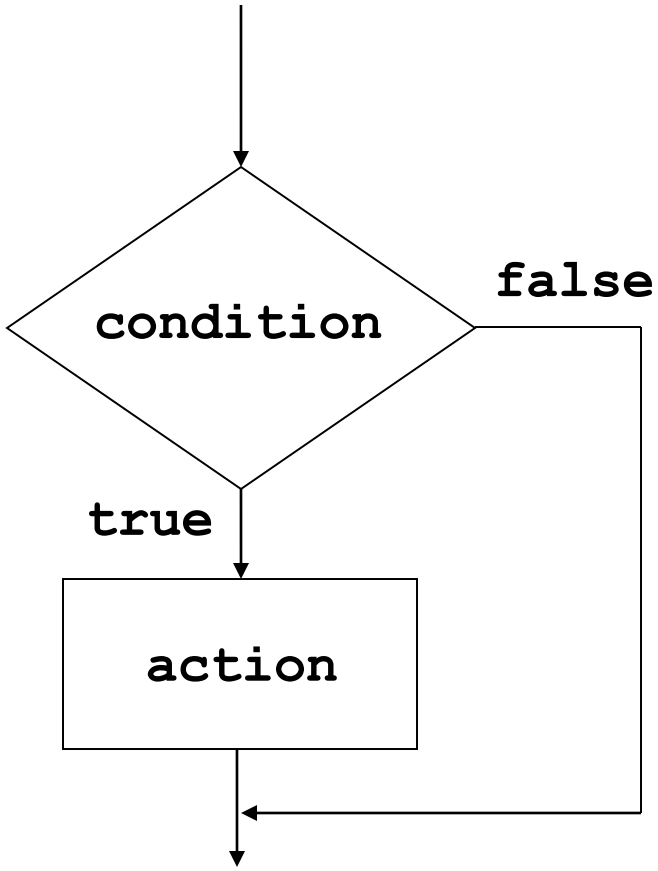
  - **if** *condition* **then**

    action1

    else

    action2

Selection structure (flow chart)



Pseudocode Primitives (continued)

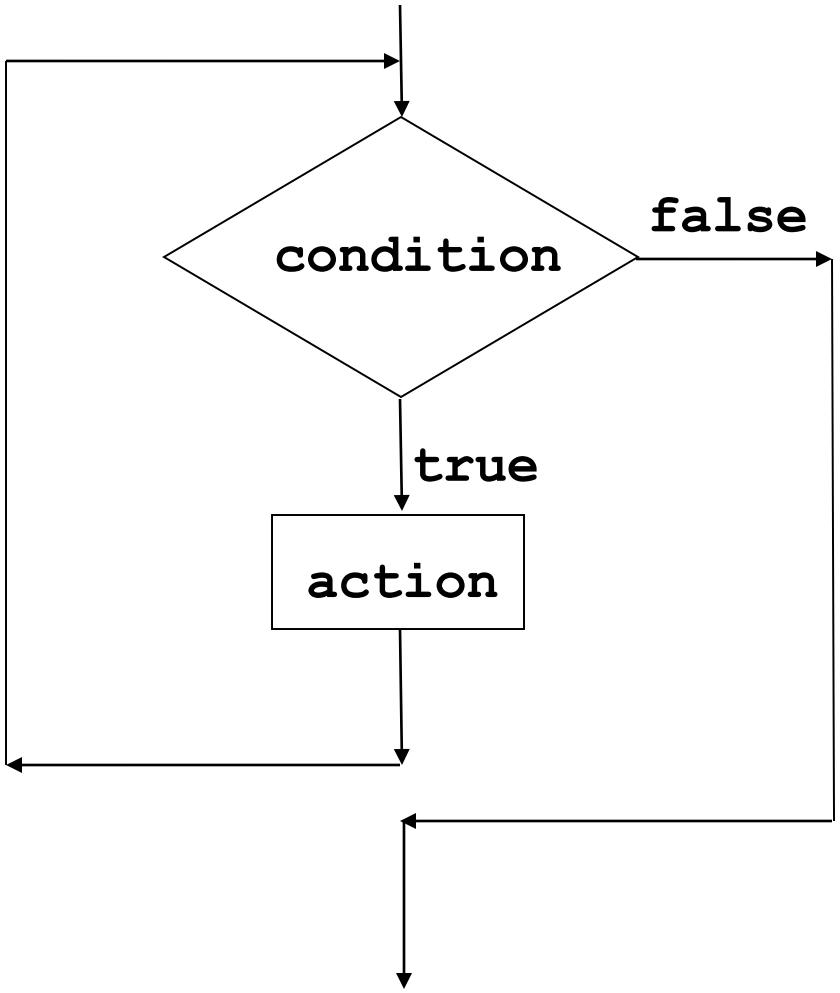
- Repeated execution

  while *condition* **do** *activity*

- Procedure

  procedure *name* (*generic names*)

Repetition structure (flow chart)



Components of repetitive control

Initialize: Establish an initial state that will be modified toward the termination condition

Test: Compare the current state to the termination condition and terminate the repetition if equal

Modify: Change the state in such a way that it moves toward the termination condition

The procedure AbsoluteValue in pseudocode

```
procedure AbsoluteValue
get Number
if (Number < 0) then
        set Number to (Number * -1)
show Number
```

The procedure IsPositive in pseudocode

```
procedure IsPositive
get Number
if (Number > 0) then
        show "positive"
else
        show "non-positive"
```

The procedure WhatIsIt in pseudocode

```
procedure WhatIsIt
get Number
if (Number > 0) then
        show "positive"
else if (Number < 0) then
        show "negative"
else
        show "neutral"
```

The procedure CountDown in pseudocode

```
procedure CountDown
set Count to 3
while (Count > 0) do
        show Count
        decrement Count
```

Problem Solving definitions

- **The process of working through details of a problem to reach a solution**[1]

Problem solving may include mathematical or systematic operations and can be a gauge of an individual's critical thinking skills

- **What you do when you don't know what to do**[2]

1)(http://www.businessdictionary.com/definition/problem-solving.html#ixzz2hjs2D3w6)
2)(http://www.teachingprofessor.com/articles/teaching-and-learning/problem-solving-a-simple-definition)

Problem Solving Strategies

- **Ask questions!**
  - *What do I know about the problem?*
  - *What is the information that I have to process in order the find the solution?*
  - *What does the solution look like?*
  - *What sort of special cases exist?*
  - *How will I recognize that I have found the solution?*

Problem Solving Strategies

- **Reuse solutions. Never reinvent the wheel!**
  - Similar problems come up again and again in different guises
  - A good problem solver recognizes a task or subtask that has been solved before and plugs in the solution

Problem Solving Strategies

- **Divide and Conquer!**
  - Break up a large problem into smaller units and solve each smaller problem
  - Applies the concept of abstraction
  - The divide-and-conquer approach can be applied over and over again until each subtask is manageable

Polya's Problem Solving Steps (1945)

- 1. Understand the problem.
- 2. Devise a plan for solving the problem.
- 3. Carry out the plan.
- 4. Evaluate the solution for accuracy and its potential as a tool for solving other problems.

Getting a Foot in the Door

- Try working the problem backwards
- Solve an easier related problem
  - Relax some of the problem constraints
  - Solve pieces of the problem first (bottom up methodology)
- Stepwise refinement (divide and conquer): Divide the problem into smaller problems (top-down methodology)

Example: Ages of Children Problem

- Person A is charged with the task of determining the ages of B's three children.
  - B tells A that the product of the children's ages is 36.
  - A replies that another clue is required.
  - B tells A the sum of the children's ages.
  - A replies that another clue is needed.
  - B tells A that the oldest child plays the piano.
  - A tells B the ages of the three children.
- How old are the three children?

How did Person A do that?

- Clearly, Person A must be…

What do we know?

a. Triples whose product is 36

(1,1,36)	(1,6,6)
(1,2,18)	(2,2,9)
(1,3,12)	(2,3,6)
(1,4,9)	(3,3,4)

b. Sums of triples from part (a)

1 + 1 + 36 = 38	1 + 6 + 6 = 13
1 + 2 + 18 = 21	2 + 2 + 9 = 13
1 + 3 + 12 = 16	2 + 3 + 6 = 11
1 + 4 + 9 = 14	3 + 3 + 4 = 10